0084. Largest Rectangle in Histogram

d | Category: Stack | LeetCode: https://leetcode.com/problems/largest-rectangle-

Problem

- Given bar heights (width 1 each), return the area of the largest rectangle inside the histogram.
- Example: [2,1,5,6,2,3] → 10 (bars 5 and 6, height 5, width 2).

Intuition

- For each bar, the best rectangle using its full height extends left and right until a shorter bar.
- A monotonic increasing stack finds both boundaries: when a shorter bar arrives, every taller bar popped has its right boundary = current index and left boundary = the new stack top.

Brute Force

- For every pair (i, j) track the min height: O(n^2).

Optimal Approach

- Iterate i from 0..n (use height 0 at i = n to flush).
- While stack non-empty and h[i] < h[top]: height = h[pop]; left = stack empty ? -1 : top; width = i - left - 1; update best.
- Push i.

Worked Example

```
[2,1,5,6,2,3].

i=1 h=1: pop 2 -> 2*1=2
i=4 h=2: pop 6 -> 6*1=6 ; pop 5 -> 5*2=10
i=6 h=0: pop 3 -> 3*1=3 ; pop 2(i4) -> 2*4=8 ; pop 1 -> 1*6=6
best = 10
```

Java Solution

```java
public class LargestRectangleInHistogram {
    public int largestRectangleArea(int[] h) {
        Deque<Integer> st = new ArrayDeque<>();
        int best = 0, n = h.length;
        for (int i = 0; i <= n; i++) {
            int cur = i == n ? 0 : h[i];
            while (!st.isEmpty() && cur < h[st.peek()]) {
                int height = h[st.pop()];
                int left = st.isEmpty() ? -1 : st.peek();
                best = Math.max(best, height * (i - left - 1));
            }
            st.push(i);
        }
        return best;
    }
}
```

Complexity

- Time: O(n). Space: O(n).

Pitfalls

- Width uses the new top after popping, not the popped index.
- The sentinel 0 at the end flushes remaining bars.